Trabalho de Pesquisa Operacional

Grupo 16

INTEGRANTES:
Nome: Gabriel Cezar Oliveira Silva
Nome: Lucas Henrique Haboski da Silva
Nome: Ryann Gustavo Valim Luiz

1. Cenário e dataset

Problema identificado

Determinar a rota de menor tempo de deslocamento entre o centro de distribuição e o endereço do cliente, dentro da malha viária de um bairro de Curitiba, em uma operação de entregas urbanas.

Fonte dos dados

OpenStreetMap, base cartográfica colaborativa de uso aberto sob licença Open Database License (ODbL). Os dados são obtidos pela biblioteca OSMnx, que consulta a Overpass API e devolve a malha viária já estruturada como grafo dirigido.

Coleta dos dados

O nome da região é enviado ao Nominatim, geocodificador do OpenStreetMap, que devolve o polígono correspondente. Esse polígono vira uma consulta à Overpass API, e a OSMnx monta o grafo a partir da resposta. O resultado é exportado em duas tabelas CSV, uma de nós e outra de arcos.

Estrutura do dataset

Cada nó representa um cruzamento ou extremidade de via, com latitude, longitude e número de ruas incidentes. Cada arco representa um trecho de rua entre dois cruzamentos, com nome da via, classificação, indicação de mão única, comprimento em metros, velocidade permitida em km/h e tempo de travessia em segundos.

Recorte adotado

Bairro Batel, em Curitiba. A malha completa da cidade passa de cem mil arcos, volume que impede qualquer inspeção manual dos dados. O recorte por bairro resulta em 166 nós e 326 arcos, o que permite conferir cada valor e validar o resultado à mão.

Adequação ao problema

O dado já chega no formato exigido pelo algoritmo, que é um grafo dirigido e ponderado. Os pesos são tempos de travessia e portanto nunca negativos, condição necessária para o uso do Dijkstra. O grafo também preserva o sentido das vias, de modo que a rota encontrada é efetivamente dirigível e não apenas uma ligação geométrica entre dois pontos.

Dados coletados

A coleta foi executada em 11 de novembro de 2026 e os arquivos resultantes estão versionados junto ao código. Os números abaixo foram medidos nesses arquivos.

Medida	**Valor**
Nós (|V|)	166
Arcos (|A|)	326
Extensão total da malha	37,1 km
Grau médio dos nós	3,93
Ruas nomeadas distintas	46
Arcos de mão única	192
Arcos de mão dupla	134

Por classificação de via, a malha tem 127 arcos residenciais, 89 secundários, 51 primários, 42 terciários, 9 de corredor de ônibus e 8 de ligação. A distribuição dos pesos e das dimensões de cada arco é a seguinte:

Medida	**length (m)**	**speed_kph (km/h)**	**travel_time (s)**
Mínimo	7,50	20,00	0,48
1º quartil	65,10	30,00	5,32
Mediana	119,37	40,00	10,00
3º quartil	148,17	50,00	14,78
Máximo	405,93	60,00	48,71
Média	113,78	40,18	10,89
Desvio padrão	66,89	9,48	7,38

O menor peso observado é 0,48 segundo. Como todos os valores de travel_time são positivos, a hipótese de não negatividade exigida pelo algoritmo de Dijkstra está verificada no dataset, e não apenas assumida.

Origem das velocidades

A velocidade de cada arco tem duas procedências. Quando o OpenStreetMap registra o campo maxspeed no trecho, esse valor é usado diretamente: é o caso de 202 dos 326 arcos, ou 62 por cento da malha. Nos 124 arcos restantes, a velocidade é atribuída pela classificação da via, a partir de uma tabela baseada nos limites urbanos brasileiros, que vai de 20 km/h em vias de serviço a 90 km/h em vias expressas, com 40 km/h como valor padrão para classificações fora da tabela.

Explicitar essa tabela não é apenas conveniência de implementação. A função de cálculo de velocidades da biblioteca interrompe a execução quando nenhum trecho da região possui maxspeed preenchido, situação comum em áreas mapeadas de forma incompleta. Informar a tabela torna a coleta reprodutível em qualquer recorte e deixa o critério auditável, já que é possível dizer de onde veio cada velocidade.

Cenário adotado

Os endereços de origem e destino foram geocodificados e associados ao cruzamento mais próximo da malha. Os três nós pertencem à componente fortemente conexa principal, que contém 163 dos 166 nós, de modo que existe caminho dirigível entre a origem e cada destino.

Papel	**Endereço**	**Nó no OpenStreetMap**
Origem (s)	Shopping Pátio Batel	11220213870
Destino (t_1)	Rua Desembargador Motta	1386657461
Destino (t_2)	Rua Olavo Bilac	324728738

Malha extraída

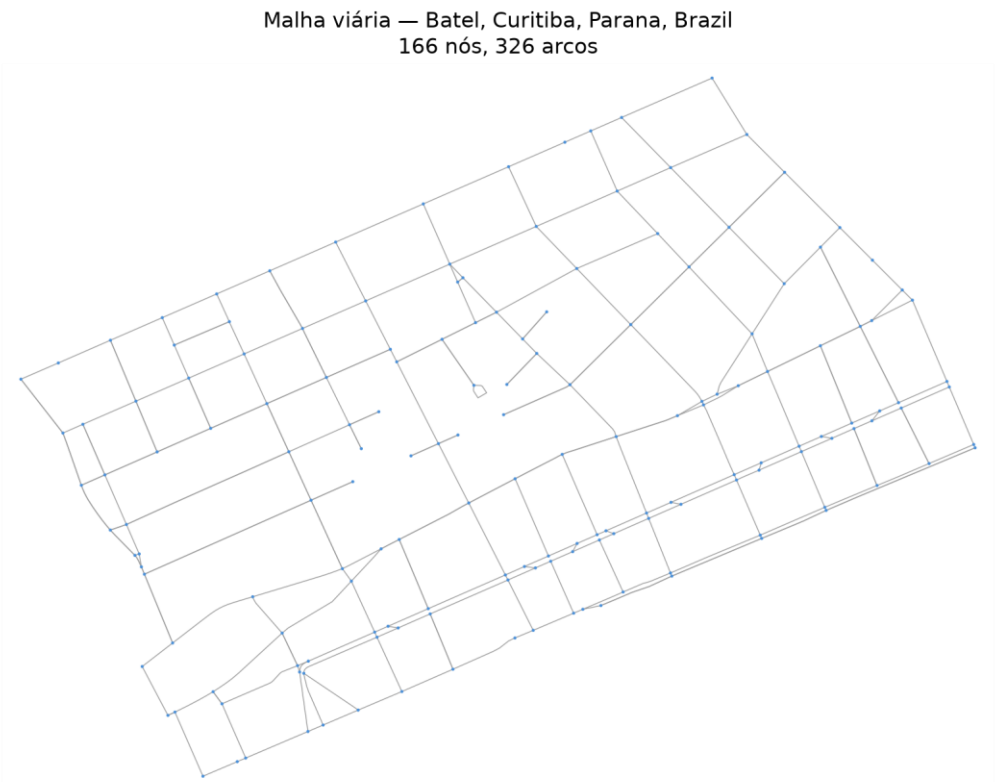


Figura 1 — Malha viária do bairro Batel, com 166 nós e 326 arcos. Dados: OpenStreetMap (ODbL).

2. Algoritmo escolhido

Algoritmo

Dijkstra, para caminho mínimo de origem única.

Justificativa

O problema é ponto a ponto, com uma origem fixa e um destino fixo, que é exatamente o caso resolvido pelo Dijkstra. Todos os pesos são tempos de viagem e satisfazem a condição de não negatividade exigida pelo algoritmo. O grafo é esparso, já que cada cruzamento liga a três ou quatro vias, o que mantém a execução rápida mesmo em malhas grandes. Por fim, o algoritmo é exato, ou seja, a rota devolvida é comprovadamente a de menor custo e não uma aproximação.

Alternativas descartadas

Bellman-Ford seria necessário apenas se houvesse pesos negativos, o que não ocorre com tempos de viagem, e possui custo maior. O A* exigiria uma heurística de distância e só traria ganho relevante em malhas bem maiores. Caixeiro viajante e fluxo máximo respondem a outras perguntas, de sequência de visitas e de capacidade da rede, que não correspondem ao problema tratado aqui.

3. Funcionamento do algoritmo

Ideia geral

O algoritmo mantém, para cada nó, um rótulo com o menor tempo conhecido até ali partindo da origem, e um ponteiro para o nó anterior no melhor caminho encontrado. Esses rótulos começam imprecisos e vão sendo corrigidos até se tornarem definitivos.

Passo a passo

1. Inicialização: O rótulo da origem recebe zero e todos os demais recebem infinito. Todos os nós entram em uma fila de prioridade ordenada pelo rótulo.

2. Extração: Retira-se da fila o nó de menor rótulo entre os ainda abertos. Esse rótulo passa a ser considerado definitivo.

3. Relaxamento: Para cada vizinho do nó retirado, verifica-se se chegar até ele passando pelo nó atual resulta em tempo menor do que o rótulo que ele já possui. Em caso afirmativo, o rótulo é atualizado e o ponteiro de antecessor passa a apontar para o nó atual.

$$d[v] \leftarrow \min\big(d[v],\ d[u] + w_{uv}\big)$$

4. Parada: Os passos 2 e 3 se repetem até que o destino seja retirado da fila. Nesse momento, seu rótulo é o tempo mínimo procurado.

5. Reconstrução: Percorrendo os ponteiros de antecessor do destino até a origem e invertendo a sequência, obtém-se a rota.

Justificativa da escolha gulosa

O Dijkstra decide de forma gulosa ao fechar sempre o nó aberto de menor rótulo, sem reavaliar decisões anteriores. Essa escolha é correta porque os pesos são não negativos. Qualquer caminho alternativo até um nó já fechado teria de passar por um nó de rótulo maior e somar arestas que apenas aumentam o custo, de modo que nenhum caminho futuro pode melhorar o rótulo fechado. Por esse motivo o algoritmo é guloso e exato ao mesmo tempo, e não precisa de retrocesso.

Custo computacional

Com fila de prioridade implementada em heap binário, o custo de execução é dado por:

$$O\big((|V| + |A|)\ \log |V|\big)$$

onde V é o número de nós e A o número de arcos. Na escala adotada a execução leva frações de segundo.

4. Modelagem

Conjuntos e parâmetros

V: conjunto dos nós, que representam os cruzamentos da malha viária.

A: conjunto dos arcos dirigidos, que representam os trechos de rua.

s: nó de origem, correspondente ao centro de distribuição.

t: nó de destino, correspondente ao endereço do cliente.

L: comprimento do arco, em metros.

v: velocidade permitida no arco. O OpenStreetMap registra esse valor em quilômetros por hora; o modelo o converte para metros por segundo, dividindo por 3,6, antes de calcular o custo.

w: custo do arco, definido como o tempo de travessia em segundos.

$$w_{ij} = \frac{L_{ij}}{v_{ij}} \geq 0 \qquad \forall\,(i,j) \in A$$

Variável de decisão

Para cada arco do conjunto A define-se uma variável binária, que indica se aquele arco faz parte da rota escolhida.

$$x_{ij} = \begin{cases} 1, & \text{se o arco } (i,j) \text{ faz parte da rota} \\ 0, & \text{caso contrário} \end{cases} \qquad \forall\,(i,j) \in A$$

Função objetivo

Minimizar o tempo total de viagem, somando o custo de cada arco efetivamente percorrido.

$$\min\ Z = \sum_{(i,j)\in A} w_{ij}\, x_{ij}$$

Como a variável assume apenas os valores 0 e 1, o produto funciona como um interruptor: o custo de um arco só entra na soma quando aquele arco é utilizado.

Restrições

R1: Saída da origem. A diferença entre o que sai e o que entra no nó de origem deve ser igual a uma unidade.

$$\sum_{j\,:\,(s,j)\in A} x_{sj} \;-\; \sum_{j\,:\,(j,s)\in A} x_{js} \;=\; 1$$

R2: Chegada no destino. A diferença entre o que sai e o que entra no nó de destino deve ser igual a menos uma unidade.

$$\sum_{j\,:\,(t,j)\in A} x_{tj} \;-\; \sum_{j\,:\,(j,t)\in A} x_{jt} \;=\; -1$$

R3: Conservação de fluxo. Em todo nó intermediário, isto é, diferente da origem e do destino, a quantidade que entra deve ser igual à quantidade que sai.

$$\sum_{j\,:\,(i,j)\in A} x_{ij} \;-\; \sum_{j\,:\,(j,i)\in A} x_{ji} \;=\; 0 \qquad \forall\, i \in V \setminus \{s,t\}$$

R4: Domínio. Cada variável assume apenas valores binários, de modo que um arco entra inteiro na rota ou não entra.

$$x_{ij} \in \{0,\,1\} \qquad \forall\,(i,j) \in A$$

Forma compacta do modelo

As três restrições de fluxo podem ser escritas em uma única expressão, atribuindo a cada nó um termo independente:

$$\sum_{j\,:\,(i,j)\in A} x_{ij} \;-\; \sum_{j\,:\,(j,i)\in A} x_{ji} \;=\; b_i \qquad \forall\, i \in V$$

em que o termo independente vale:

$$b_i \;=\; \begin{cases} 1, & \text{se } i = s \\ -1, & \text{se } i = t \\ 0, & \text{caso contrário} \end{cases}$$

Nessa forma fica evidente que o caminho mínimo é um caso particular do problema de fluxo de custo mínimo, com uma única unidade de fluxo. A matriz de incidência entre nós e arcos de um grafo dirigido é totalmente unimodular, o que faz com que a relaxação linear da restrição de integralidade já produza solução inteira. Por esse motivo o problema dispensa métodos de programação inteira como branch and bound, e pode ser resolvido diretamente por um algoritmo combinatório em tempo polinomial.

Critério de custo adotado

O projeto utiliza o tempo de travessia como custo, por ser a grandeza que a operação de entrega assume como compromisso com o cliente. O comprimento do arco é utilizado como critério de comparação, executando o mesmo modelo com o comprimento no lugar do tempo, de modo a medir quanto se ganha em relação à rota de menor distância.

Múltiplos destinos

O modelo acima descreve um único par origem destino. O cenário adotado tem dois destinos, e isso não altera a formulação. O algoritmo de Dijkstra é de origem única: uma execução a partir de s produz o rótulo de caminho mínimo para todos os nós da malha simultaneamente, e interromper a busca quando t é retirado da fila é apenas uma otimização de parada antecipada. Dois clientes atendidos a partir do mesmo centro de distribuição são, portanto, duas aplicações independentes do mesmo modelo, e não um problema de sequenciamento de visitas, que corresponderia ao caixeiro viajante, descartado na seção 2.

Solução esperada

O conjunto de arcos cuja variável vale 1 forma a rota ótima, e o valor da função objetivo corresponde ao tempo total de deslocamento. Espera-se que a rota de menor tempo percorra distância maior que a rota de menor distância, por privilegiar vias de velocidade mais alta, e que as duas sejam visivelmente distintas quando sobrepostas ao mapa do bairro.

5. Referências

Fontes de dados

OPENSTREETMAP. Mapa de Curitiba, Paraná, Brasil. Coordenadas: -25.4296, -49.2713. Disponível em: https://www.openstreetmap.org/#map=13/-25.4296/-49.2713. Acesso em: 11 nov. 2026.

OPEN DATA COMMONS. Open Data Commons Open Database License (ODbL) v1.0. Disponível em: https://opendatacommons.org/licenses/odbl/1-0/. Acesso em: 11 nov. 2026.

OPENSTREETMAP FOUNDATION. Attribution Guidelines. Disponível em: https://osmfoundation.org/wiki/Attribution. Acesso em: 11 nov. 2026.

Ferramenta de coleta

BOEING, G. OSMnx: New Methods for Acquiring, Constructing, Analyzing, and Visualizing Complex Street Networks. Computers, Environment and Urban Systems, v. 65, p. 126-139, 2017. DOI: 10.1016/j.compenvurbsys.2017.05.004.

OSMNX. Documentação oficial da biblioteca. Disponível em: https://osmnx.readthedocs.io. Acesso em: 11 nov. 2026.

Referência do algoritmo

DIJKSTRA, E. W. A note on two problems in connexion with graphs. Numerische Mathematik, v. 1, p. 269-271, 1959. DOI: 10.1007/BF01386390.